Tape-Free Adjoint Greeks on FPGAs: Heston-Model Option Sensitivities with Per-Greek Rounding-Error Bounds

Yash Agrawal
Birla Institute of Technology and Science, Pilani
Pilani, India

Govind Prasad
Birla Institute of Technology and Science, Pilani
Pilani, India

***Abstract*—Option Greeks, the sensitivities of a price to its inputs, drive hedging and risk limits, and on hardware they are still computed by bump-and-reprice: one extra pricing per input, at an accuracy that depends on an arbitrary bump size. This paper presents the first FPGA engine that computes Greeks by adjoint (reverse-mode) differentiation. It prices a European option under the Heston stochastic-volatility model with the COS Fourier-cosine method and returns the price and all nine first-order Greeks in one pass. Because the COS pricer performs the same operations for every input, its reverse sweep is compiled into the same statically scheduled fixed-point datapath as the forward pass, with no tape memory or replay logic, and fills multiplier slots the forward pass leaves idle. A generator produces, from one operation graph, a bit-accurate model, a first-order rounding-error bound for the price and each Greek, a modulo-scheduled Verilog datapath and the host code. On a Zynq-7020 the price and nine Greeks take 4,733 cycles, 1.04 times the price alone and 18.4 times fewer than bump-and-reprice on the same hardware; the adjoint Greeks are 32–560 times more accurate than the best bump-and-reprice. The bound was never exceeded over 21,000 test inputs (worst 0.378 of the bound), and the engine ran bit-exact on silicon on two FPGA families: 450/450 results on a ZedBoard and 1,010/1,010 on a Zynq UltraScale+ ZCU102, including 50 calibrations to real NIFTY option data. On the ZCU102 an evaluation takes 15.9 μs, 4.9–15 times faster than forward-mode AD, taped adjoint AD and bump-and-reprice on a laptop core and 1.3 times faster than hand-derived analytic Greeks.**

***Index Terms*—FPGA, adjoint algorithmic differentiation, option Greeks, Heston model, COS method, fixed-point arithmetic, rounding-error analysis, high-level synthesis**

I. Introduction

A trading desk needs not only the prices of the options it holds but their Greeks, the partial derivatives of each price with respect to spot, strike, maturity, rate and the model's parameters. Greeks set hedge ratios and risk limits and feed model calibration, so they are recomputed whenever the market moves. Under the Heston stochastic-volatility model [1] the price has no closed form and is computed numerically, here by the COS method [2].

The usual way to obtain Greeks, in software and in every FPGA design we know of, is *bump-and-reprice*: perturb one input and price again. Nine Greeks by central differences cost 19 pricings, and the result depends on a bump size for which no setting is safe for every Greek (Fig. 8). Software replaced this with *adjoint algorithmic differentiation* (AAD) [4], [5], which obtains the derivatives of one output with respect to all inputs in a single reverse sweep. In software, however, the reverse sweep reads a *tape* of the forward computation; on our CPU baseline taped AAD costs 7.6 pricings and forward-mode AD 5.7. No FPGA work has computed Greeks by an adjoint [6], [11]–[13].

This paper shows that, for a Fourier pricer, the adjoint maps onto hardware unusually well. The COS method performs exactly the same operations for every input, so the lifetime of every forward value is known at design time and the reverse sweep can be scheduled statically, alongside the forward pass, on shared arithmetic units. The contributions are:

1) **A tape-free adjoint datapath** for Heston–COS Greeks: the reverse sweep is compiled into the forward pass's fixed-point datapath with no tape memory or replay logic (Section III). To our knowledge this is the first FPGA computation of option Greeks by adjoint differentiation, and the first FPGA implementation of the COS method.
2) **All nine Greeks for 1.04 times the cost of the price**: the reverse sweep fills multiplier slots left idle by the CORDIC-bound forward pass. Against forward-mode datapaths built by the same generator, the adjoint needs the fewest multipliers for the same throughput (Section VI-B).
3) **A first-order rounding-error bound for every Greek** of a fixed-point datapath that itself performs the adjoint, computed from the same operation graph as the hardware and never exceeded over 21,000 inputs and on silicon (Section VI-D).
4) **A generator** that emits, from one description, the bit-accurate model, the bound, the modulo schedule, the Verilog, the bus wrappers and the host C code, and **bit-exact silicon results on two FPGA families** (Section VI-G).

We state plainly what is not claimed: the reverse sweep is derived by hand in the generator's graph, not by an AD tool; the bound is first order and covers rounding only; and on the small FPGA a CPU core with hand-derived analytic Greeks is faster (Section VII).

II. Background and Related Work

A. Heston model and the COS method

Under the risk-neutral measure the Heston model is $\mathrm{d}S_t = rS_t\,\mathrm{d}t + \sqrt{v_t}S_t\,\mathrm{d}W_1$, $\mathrm{d}v_t = \kappa(\theta - v_t)\,\mathrm{d}t + \xi\sqrt{v_t}\,\mathrm{d}W_2$, $\mathrm{d}W_1\mathrm{d}W_2 = \rho\,\mathrm{d}t$, with nine pricing inputs $p = (S_0, K, T, r, v_0, \kappa, \theta, \xi, \rho)$. The characteristic function φ of the log-price is known in closed form; we use the form of Albrecher et al. [3], which avoids the complex-logarithm discontinuity. The COS method [2] expands the density on a truncation range $[a, b]$ in a cosine series, giving for a European put

$$V_{\mathrm{put}} \approx Ke^{-rT} \sum_{k=0}^{N-1}{}' \mathrm{Re}\Big[\varphi(u_k)\, e^{\,iu_k(x-a)}\Big]\, U_k, \quad u_k = \tfrac{k\pi}{b-a}, \tag{1}$$

with $x = \ln(S_0/K)$, closed-form payoff coefficients U_k, and $\sum'$ halving the first term. Calls follow from put–call parity, $V_{\mathrm{call}} = V_{\mathrm{put}} + S_0 - Ke^{-rT}$. The N terms are *independent*, which the hardware pipelines.

B. Computing Greeks

Bump-and-reprice approximates $\partial V/\partial p_j$ by $[V(p + he_j) - V(p - he_j)]/2h$: 19 pricings for nine Greeks, with a truncation error $O(h^2)$ and a rounding error $O(\epsilon/h)$, so the error is U-shaped in h. *Forward-mode* AD carries a tangent per input direction. *Reverse-mode* (adjoint) AD propagates $\bar{v} = \partial V/\partial v$ backwards from the output and returns all nine derivatives in one sweep at a small constant multiple of the pricing cost [7]; it is standard for Greeks in software [4], [5]. For the Heston–Fourier price, Cui et al. [9] derived the gradient by hand.

C. Hardware option pricing

Published FPGA finance work is mostly Monte Carlo pricing reported as batch runtime or energy against multi-core servers. Table I positions this work; the reported figures measure different things and are not directly comparable. None of the hardware works computes Greeks by an adjoint, none uses a Fourier method, and none gives a per-Greek accuracy guarantee for its fixed-point arithmetic.

III. Method

A. Why the adjoint needs no tape here

In software, a reverse sweep needs the forward values in reverse order, which is why AAD tools record a tape. In (1) the control flow does not depend on the input: every term runs the same operations, the truncation range is fixed per evaluation, and there are no data-dependent branches. The forward and reverse computations of one term therefore form a fixed operation graph whose every value has a lifetime known at design time. The scheduler keeps each forward value in a register for exactly as long as the reverse sweep needs it (447 storage registers per term in the Zynq-7020 design), so the "tape" is compiled into the schedule and costs no memory traffic.

The Greeks returned are the derivatives of the COS approximation itself, including the put–call parity terms, with $[a, b]$ held fixed (Section V-A). Per term the reverse sweep propagates adjoints from the term's contribution back to the eight model inputs, reusing the forward pass's reciprocals; a power-of-two normalisation keeps the small adjoints of high-frequency terms at full precision; and nine accumulators sum the price and the Greek contributions.

B. Arithmetic and fixed point

The datapath has no dividers. Reciprocals and square roots use Newton–Raphson iterations from table seeds, the exponential uses Tang's method [15], the logarithm a table and a short series, and sines, cosines and complex magnitudes and angles CORDIC [16]. Each COS term costs exactly 250 multiplications, three CORDIC rotations and two CORDIC vectorings (126 multiplications for the price alone). Values are two's-complement fixed point with FL fraction bits (56-bit words with $FL = 28$ on the Zynq-7020, 64-bit with $FL = 32$ on UltraScale+), rounded to nearest after every operation. Data-dependent shifts are built only as wide as the ranges measured over the input domain with margin, which cut logic by 35%; an input that would exceed a measured range raises a `range_err` flag rather than return a wrong value.

C. First-order rounding-error bound

Let δ_i be the rounding error introduced at operation i, with $|\delta_i| \le 2^{-(FL+1)}$ for round-to-nearest. To first order the error in an output y is $\sum_i (\partial y/\partial w_i)\,\delta_i$, where w_i is the result of operation i, so

$$|\hat{y} - y| \;\le\; \sum_i \left|\frac{\partial y}{\partial w_i}\right| 2^{-(FL+1)} \;=:\; B_y, \tag{2}$$

Linnainmaa's first-order analysis [8]. The sensitivities $\partial y/\partial w_i$ are themselves obtained by a reverse sweep over the operation graph, which here includes the hardware's own adjoint operations, so B_y bounds the rounding error of each Greek as the hardware computes it. B_y is computed in software from the same graph (0.1 s per parameter set); the engine does not output it. Products of rounding errors are dropped; they are about 2^{-FL} times smaller, and two checks in Section VI-D confirm this.

D. The generator and modulo scheduling

Fig. 1 shows the flow. One Python description of setup (once per evaluation), term ($N = 128$ times) and finish (once) is an operation graph from which the generator derives the bit-accurate fixed-point model, a double-precision model, the bound (2), and a modulo schedule [17] that is emitted as Verilog. Because the RTL and the model come from the same graph, the testbench compares every register at the cycle it becomes valid (1,213–1,962 checks per design), and every property shown for the model holds for the hardware.

The terms are independent, so a new term starts every initiation interval (II) while earlier terms are in flight (Fig. 2). With 8 shared multipliers, 3 CORDIC rotators and 2 vectorers, all three resource classes give the same interval, $\lceil 250/8 \rceil = 3{\cdot}32/3 = 2{\cdot}32/2 = 32$ cycles, and an evaluation takes

TABLE I
POSITIONING AGAINST RELATED WORK. REPORTED RESULTS ARE EACH PAPER'S OWN HEADLINE FIGURES AND ARE NOT DIRECTLY COMPARABLE.

Work	Platform	Model / pricing	Greeks computed by	Accuracy guarantee	Reported result
de Schryver et al. [12]	FPGA (Xilinx)	Heston, Monte Carlo	— (price only)	none	energy-efficient pricing
Klaisoongnoen et al. [11]	Alveo U280, 300 MHz	STAC-A2: Heston MC, Longstaff–Schwartz	re-simulation (finite differences)	none (precision explored)	8–185× efficiency vs. 2× Xeon 8260M
AMD Vitis Quant. Library [13]	Alveo (HLS)	Heston MC; Heston closed form	central differences; price only	none	library
Gremse et al. [14]	GPU	general	taped adjoint AD	none	AAD on GPUs
Cui et al. [9]	CPU	Heston, Fourier	analytic gradient (by hand)	—	≈10× faster than numerical gradient
This work	Zynq-7020, Zynq US+	Heston, COS (N=128), 56/64-bit fixed point	**tape-free adjoint datapath**, all 9 in one pass	**first-order bound per Greek**	1.04× price cost; 15.9 μs; bit-exact on silicon

TABLE II
ENGINE CONFIGURATIONS (CYCLES FROM CYCLE-EXACT RTL SIMULATION)

Config.	Word / frac.	Mult.	On chip	Cycles
`z7`	56 / 28	8	all	4,733
`z7h`	56 / 28	8	128-term loop	4,572
`zu`	64 / 32	32	all	1,593

$127 \cdot 32 + 506 + 2 = 4{,}572$ cycles for the loop. Because the CORDIC units set the pace, the forward pass leaves multiplier slots idle in every 32-cycle window; the reverse sweep's 124 extra multiplications fill them, which is why the Greeks cost 4%.

TABLE III
IMPLEMENTATION RESULTS (VIVADO, POST-ROUTE, JTAG TEST SYSTEMS)

	ZedBoard	ZCU102	ZCU104
Device	xc7z020-1	xczu9eg-2	xczu7ev-2
Engine	`z7h`	`zu`	`zu`
LUT	38,752 (72.8%)	77,515 (28.3%)	77,863 (33.8%)
FF	32,134 (30.2%)	63,547 (11.6%)	63,796 (13.8%)
DSP	72 (32.7%)	510 (20.2%)	510 (29.5%)
BRAM tiles	4.5 (3.2%)	7.5 (0.8%)	6.5 (2.1%)
Clock (MHz)	70	100	100
WNS (ns)	+0.458	+1.601	+1.469
$F_{\max}$ (MHz)	≈79[a]	≈119	≈117
Run on silicon	yes	yes	no

[a] Engine alone, out of context; the 56-bit carry chains limit the clock. Totals include the JTAG-to-AXI master and interconnect. Vivado 2025.2 (ZedBoard, ZCU104) and 2026.1 (ZCU102).

IV. IMPLEMENTATION

Three configurations come from the same generator (Table II). On the Zynq-7020 the setup and finish steps (about 100 multiplications and a few transcendental functions per evaluation) run on the host, as generated C that is bit-exact with the model, and the FPGA runs the 128-term loop; the 64-bit configuration computes everything on chip. Each engine sits behind an AXI4-Lite register file. For testing, a JTAG-to-AXI master drives the register file from the host with no processor software, so the test exercises the datapath alone. Table III gives the implementation results in Vivado.

V. EXPERIMENTAL METHODOLOGY

Input sets. (i) A 21-case grid over moneyness, maturity, vol-of-vol and correlation, calls and puts. (ii) A 10,500-input sweep: 6,000 uniform over the verified domain (S_0=100, $K \in [60, 150]$, $T \in [0.1, 3]$, $r \in [0, 0.1]$, $v_0, \theta \in [0.005, 0.25]$, $\kappa \in [0.2, 6]$, $\xi \in [0.1, 1]$, $\rho \in [-0.95, 0.6]$), 500 in each of seven hard regimes (Feller condition violated, ρ near its limits, deep strikes, very short and long maturities, high vol-of-vol, very low variance) and 1,000 with one input outside the domain. (iii) A held-out sweep of the same composition with a fresh seed. (iv) 50 board cases (2 fixed, 48 random). (v) 50 calibrations of the model to NIFTY index option quotes on 50 trading days.

References. Accuracy is measured against the same algorithm in double precision (rounding error) and, separately, the COS result against an independent Fourier-integral price (method error).

CPU baselines. The identical algorithm in C++ (double precision, g++ 13, `-O3 -march=native`) on an Intel Core i5-1155G7 laptop (4 cores, 8 threads): price only, bump-and-reprice (19 pricings), an optimised bump that reuses the characteristic function, forward-mode AD (9 directions in one pass), taped adjoint AD with CoDiPack [10], and hand-derived analytic Greeks after Cui et al. [9]. Every method matches the reference model (adjoint to 1.3×10^{-10} relative). Latency is the best of five passes over 2,000 inputs on one core of an otherwise idle machine; throughput uses eight independent processes.

A. Assumptions and their rationale

Table IV lists every modelling and measurement assumption, why it was made, and how its effect was checked.

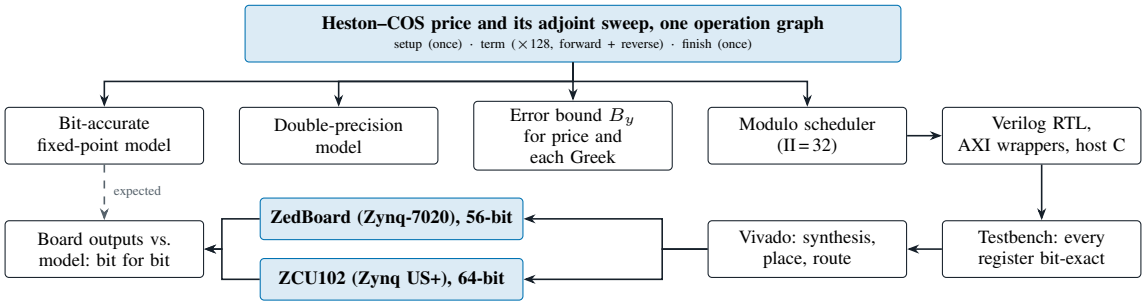


Fig. 1. Design flow. One operation graph yields the bit-accurate model, the error bound and the scheduled RTL; the RTL is checked register by register against the model, implemented in Vivado and run on two boards, whose outputs are compared bit for bit with the same model.

TABLE IV
ASSUMPTIONS, THEIR RATIONALE, AND HOW EACH WAS CHECKED

	Assumption	Why it was made	How its effect was checked or bounded
A1	Bounded input domain (Section V)	Fixed-point shifters are sized from ranges measured over a domain; an unbounded domain would need floating point. The domain covers typical equity-index calibrations.	Inputs outside raise `range_err`; held-out sweep: no in-domain flag, every unflagged out-of-domain result within its bound.
A2	S_0 normalised to 100	The price is homogeneous of degree one in (S_0, K), so one scale suffices; the host rescales inputs and outputs.	Exact identity; real NIFTY cases were rescaled this way.
A3	COS with N=128 terms and range $c_1 \pm 10\sqrt{c_2}$ (cumulants)	Standard choices for Heston–COS [2]; 128 fixes the pipeline length.	Method error measured separately against a Fourier integral (Section VI-F).
A4	Range $[a, b]$ held fixed when differentiating	$[a, b]$ is a numerical parameter, not a model input; its derivative only affects the truncated tails. The CPU reference uses the same convention.	Greeks compared with an independent Fourier-integral reference; differences are within the method error.
A5	Puts priced; calls by put–call parity	Call payoff coefficients grow like e^b and cancel badly in fixed point.	Worst call price error 75× smaller with parity.
A6	First-order bound (products of rounding errors dropped)	Second-order terms are $\approx 2^{-FL}$ smaller; a rigorous enclosure (interval arithmetic) would be far looser.	50-digit evaluation: error ≤0.24 of the bound; bound still holds at 44 bits, where second-order terms are 4,096× larger.
A7	Round-to-nearest, $\lvert\delta_i\rvert \le 2^{-(FL+1)}$, at every operation	This is what the generated RTL implements.	RTL bit-exact with the model, which applies the same rounding.
A8	Word lengths 56/64 bits	56 bits is the shortest word at which every Greek's worst relative error is $\le 1.2 \times 10^{-5}$ on the grid (52 bits: 5.8×10^{-4}), and a 56×56 multiplier maps to 9 DSP48 blocks.	Word-length sweep 44–64 bits; bound held at every length.
A9	Latency = cycle count × clock period, engine only	Over JTAG each register access takes milliseconds, so the board run checks outputs but cannot time the engine.	Cycle counts are exact (RTL simulation equals the schedule); bus transfers (19 words in, 20 out) are excluded and stated as such.
A10	FPGA power from simulated switching activity of the routed engine	A board-rail measurement was not available; Vivado's default 12.5% toggle rate underestimated dynamic power 3–6×.	Routed netlist simulated gate by gate on four bit-exact evaluations; activity read back into Vivado (confidence High).
A11	CPU package power from **[HWiNFO measurement]**	The CPU energy needs a measured power; until then the part's rated 12–28 W were used.	**[measured CPU package power, idle and under load]**
A12	Vega is $\partial V/\partial v_0$ (variance)	v_0 is the model input; $\partial V/\partial\sigma_0 = 2\sigma_0\,\partial V/\partial v_0$.	Exact conversion.
A13	Adjoint written by hand in the operation graph	Automatic derivation of the reverse sweep in the generator is future work.	Agrees with three independent forward-mode datapaths to 6×10^{-14} and with CoDiPack to 1.3×10^{-10}.
A14	NIFTY cases: calibrated parameters, $r = 0.06$	Used only as realistic inputs for the silicon test, not to evaluate trading.	All 50 inside the domain; 500/500 outputs bit-exact.

VI. RESULTS

A. Cost of the Greeks

Fig. 3 compares, on identical hardware, the price alone, the price with nine adjoint Greeks, and bump-and-reprice. On the 56-bit design the nine Greeks add 165 cycles (3.6%) to the price; bump-and-reprice needs 18.4× more cycles. On the 64-bit design, where the 32 multipliers rather than the CORDIC units set the pace, the Greeks cost 1.55 pricings and bump-and-reprice 12.3× the adjoint. Against our own first-generation, hand-written engine (433,779 cycles, which did not fit a Zynq-7020) the generated design is 92× faster.

B. Adjoint against forward mode

Three forward-mode (tangent) datapaths were built by the same generator, with the same primitives, range and scheduler: *dense* (all eight directions through every operation, as a vector

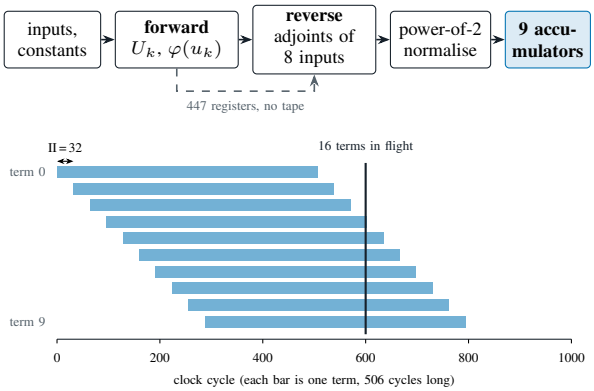


Fig. 2. Top: one COS term; the reverse sweep shares the forward pass's datapath and reads forward values from scheduled registers. Bottom: the modulo schedule (first 10 of 128 terms); 8 multipliers and 5 CORDIC units are shared by about 16 overlapping terms.

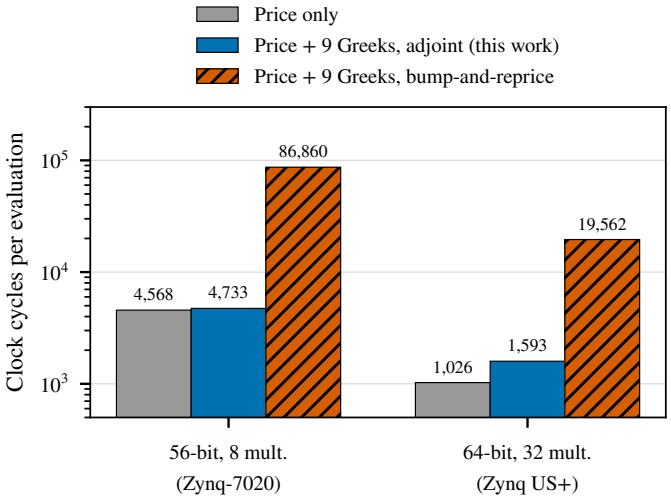


Fig. 3. Clock cycles per evaluation on the same hardware (cycle-exact RTL simulation, 128 COS terms).

forward-mode AD tool does), *sparse* (structural zeros skipped) and *factored* (derivatives of the exponent with respect to its inner complex quantities formed once per term, as a hand derivation of analytic Greeks does). All return the same price and Greeks as the adjoint to 6×10^{-14} in double precision. Fig. 4 and Table V show that the adjoint needs the fewest multiplications: 124 per term beyond the price, against 187 for the best forward method. The 1.04× cost is not unique to the adjoint: with 16 multipliers all but the dense datapath become CORDIC-bound at about 4,570 cycles. What the adjoint saves is hardware: it reaches that pace with 8 multipliers (72 DSP blocks), where the factored forward datapath needs 10 (90) and the dense one 26 (234, more than the device has).

C. Design space

Fig. 5 sweeps the number of shared 64-bit multipliers. Cycles halve as multipliers double until the dependency chain of a term dominates beyond about 64; the adjoint stays within

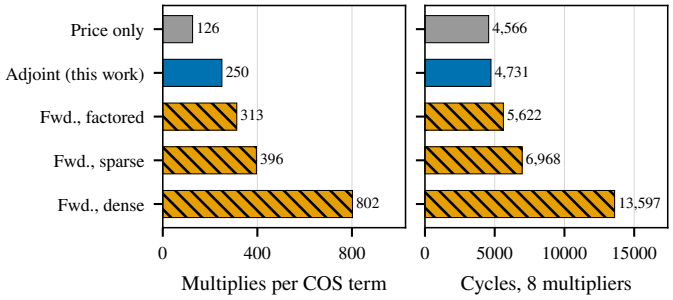


Fig. 4. Price and nine Greeks by datapath type, 56-bit design: multiplies per COS term and cycles with 8 shared multipliers.

TABLE V
CYCLES PER EVALUATION BY DATAPATH AND SHARED MULTIPLIERS

		56-bit		64-bit	
Datapath	**Mult./term**	**8**	**16**	**16**	**32**
Price only	126	4,566	4,561	1,538	1,024
Adjoint (this work)	**250**	**4,731**	4,619	**2,612**	**1,591**
Forward, factored	313	5,622	4,571	3,069	1,795
Forward, sparse	396	6,968	4,571	3,690	2,171
Forward, dense	802	13,597	7,150	7,155	3,822

Forward datapaths omit the fixed-point rescaling of tangents a working engine would need, so their counts are lower bounds.

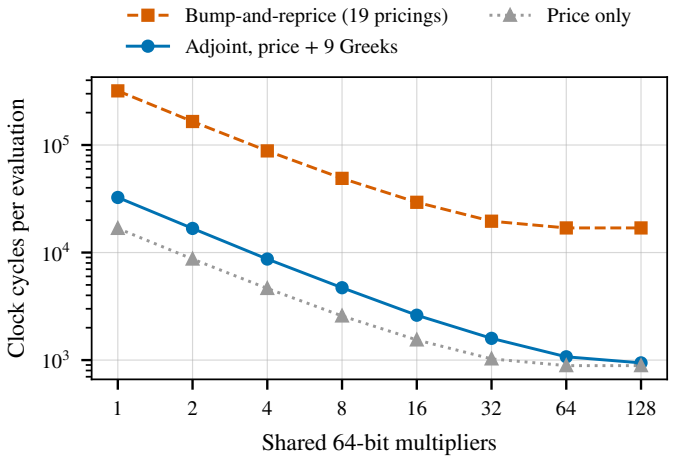


Fig. 5. Cycles against the number of shared multipliers (64-bit configuration).

1.1–1.9× of the price and 9.8–18× below bump-and-reprice throughout. The same generator thus targets a small or a large device from one parameter.

D. Accuracy and the error bound

Fig. 6 gives the worst relative error of each output over the 21-case grid against the same algorithm in double precision, and the worst error as a fraction of its bound (2). The 56-bit price is accurate to 6.1×10^{-7} and the Greeks to at most 1.2×10^{-5}; the 64-bit design gains one to two orders of magnitude. No output reaches more than 0.103 of its bound.

Over the 10,500-input sweep no unflagged result exceeded its bound (worst 0.378), and there was no overflow. On the held-out sweep (Fig. 7) none of the 9,500 in-domain inputs

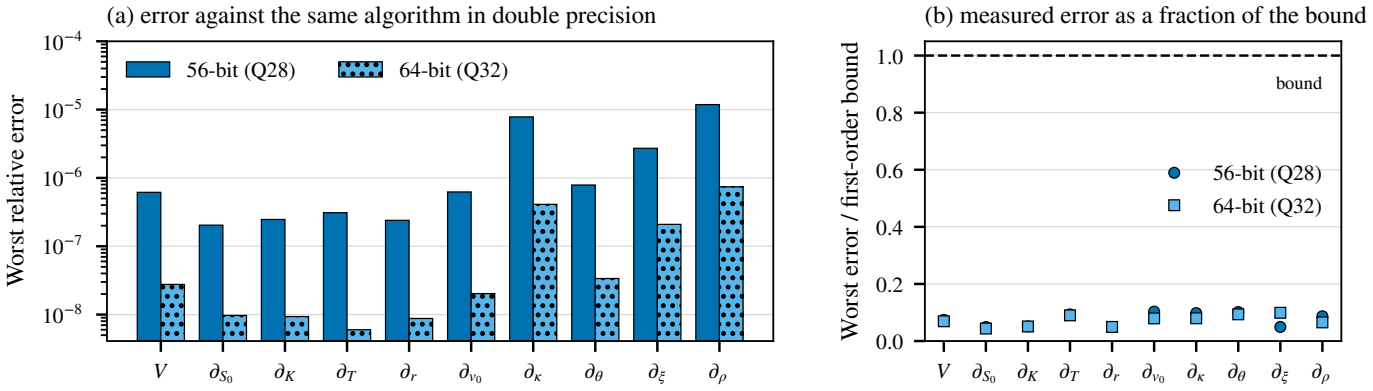


Fig. 6. Accuracy over the 21-case grid. (a) Worst relative error of the price V and each Greek against the same algorithm in double precision. (b) Worst measured error divided by the first-order bound (2); 1 would mean the bound is met with equality.

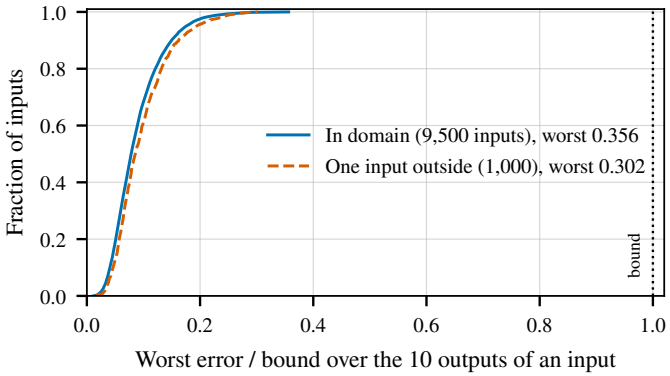


Fig. 7. Held-out 10,500-input sweep: distribution of the worst error/bound ratio over the ten outputs of each input (unflagged inputs; 56-bit design).

TABLE VI
SIGNIFICANT FIGURES, 56-BIT DESIGN, 9,500 HELD-OUT IN-DOMAIN INPUTS

Out.	Med.	P99	Worst	Out.	Med.	P99	Worst
V	7.6	5.6	4.4	∂_{v_0}	7.6	5.3	3.3
∂_{S_0}	8.1	5.7	3.9	∂_κ	6.0	3.5	2.2
∂_K	7.9	5.7	3.9	∂_θ	7.2	4.9	3.5
∂_T	7.6	5.5	4.0	∂_ξ	5.9	3.0	1.4
∂_r	7.9	5.9	4.4	∂_ρ	7.0	4.9	3.7

$-\log_{10}$ of the relative error against double precision; inputs where a Greek is within 1% of zero are excluded.

raised `range_err`, the worst in-domain ratio was 0.356, and every unflagged out-of-domain result also stayed within its bound. Two checks support the first-order assumption (A6): evaluated in 50-digit arithmetic on 100 held-out inputs, the hardware error is at most 0.24 of the bound, the same to three decimals as against double precision; and the bound still holds at 44 bits, where second-order terms are 4,096× larger. Table VI gives the significant figures delivered: the price and the market Greeks keep at least 5.3 figures for 99% of inputs, while $\partial V/\partial\kappa$ and $\partial V/\partial\xi$, small and computed through long chains with cancellation, keep 3.5 and 3.0. Each comes with its bound, so a user knows its accuracy before using it.

E. Against bump-and-reprice accuracy

Fig. 8 shows bump-and-reprice error against the relative bump size on the same pricer and word length. The curves are U-shaped, with the optimum at a different h for each Greek; the adjoint is a single point well below every curve. Over the nine Greeks the adjoint is 32–560× more accurate (median 145×) than bump-and-reprice even with the best bump size chosen afterwards for each Greek, and has nothing to tune.

F. COS method error

The bound covers rounding only. On the 50 board cases the COS result was compared with an independent Fourier-integral price: the method error has a median of 2×10^{-11} for the price but reaches 7.8×10^{-3} for vega in the worst case ($\xi = 1$, $T = 0.34$), where 128 terms are too few (256 terms remove it), and a deep out-of-the-money put needs a 1.5× wider range. The total error against the Heston model is at most the rounding bound plus this method error, and only the former is bounded. Any implementation of this COS configuration, in hardware or software, shares the method error.

G. Silicon validation

Table VII summarises the board runs. Cases ran back to back with no reset between them. Every output of every case matched the bit-accurate model, so every accuracy result above, including the bound, holds on silicon. The worst error/bound ratio on silicon was 0.255, and the worst price error against double precision 7.3×10^{-7} on the 64-bit engine.

H. Against a CPU: latency, throughput and energy

Fig. 9 and Table VIII compare the same algorithm on the FPGA and on one laptop core. The engine is faster than every general-purpose method: on the ZCU102, 4.9× faster than forward-mode AD, 6.5× than taped adjoint AD and 15× than bump-and-reprice. Against hand-derived analytic Greeks, the strongest software baseline, the ZedBoard's loop is 3.2× slower and the ZCU102's whole evaluation 1.3× faster. Four CPU cores give 2.5× the throughput of one 64-bit engine, which uses a fifth of the ZCU102's DSP blocks.

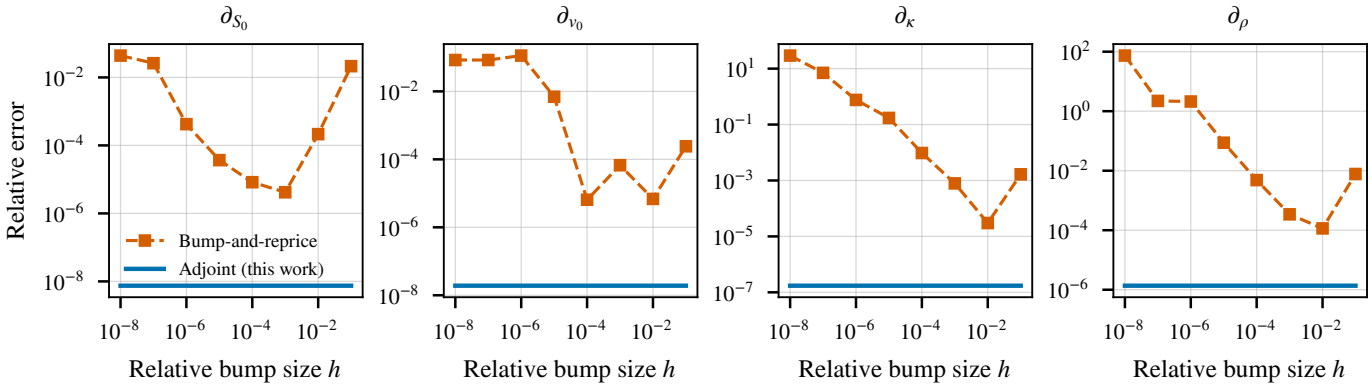


Fig. 8. Relative error of bump-and-reprice (central differences) against relative bump size, and of the adjoint engine, for four Greeks (56-bit design, base case). The other five Greeks behave alike.

TABLE VII
SILICON VALIDATION (OUTPUTS COMPARED BIT FOR BIT WITH THE MODEL)

Board	Input set	Cases	Bit-exact	Err./bound
ZedBoard	vector + sweep	1 + 50	9 + 450 / 459	0.199
ZCU102	vector	1	10 / 10	0.099
ZCU102	sweep	50	500 / 500	0.243
ZCU102	real NIFTY calibrations	50	500 / 500	0.255
Total			**1,469 / 1,469**	≤ 0.255

ZedBoard: 9 raw sums per case, finished on the host by bit-exact code. ZCU102: 10 outputs per case computed on chip. No `range_err`.

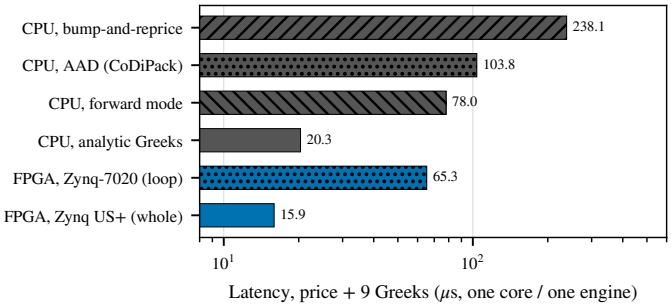


Fig. 9. Latency of one price and nine Greeks: one CPU core (Intel i5-1155G7, idle machine) and one FPGA engine (cycle count × clock period; A9).

On energy, the engine draws 0.676 W on the Zynq-7020 and 2.485 W on the 64-bit design under simulated activity, giving 44 and 40 μJ per evaluation. The CPU, at its measured package power of **[HWiNFO: P_{pkg} W]** under full load, uses **[E_{CPU} μJ]** per evaluation at its full throughput, so the FPGA uses **[ratio]**× less energy per evaluation. (With the CPU's rated 12–28 W instead of a measurement, the ratio is 2–4×.)

VII. Discussion and Limitations

Where the advantage lies. The engine's case against a CPU is the cost and the certified accuracy of its Greeks, not raw speed: the nine Greeks cost 4% over the price in hardware against 50% for the best hand-written software and 470–660% for AD tools, each Greek carries a bound, and the latency is deterministic. The latency lead over hand-derived Greeks is narrow (1.3×) and exists only on the larger device.

TABLE VIII
LATENCY, THROUGHPUT AND ENERGY PER EVALUATION (PRICE + 9 GREEKS)

Method	μs	Cost	Eval./s	Energy
CPU, price only	13.7	1×	—	—
CPU, bump-and-reprice	238.1	17.4×	—	—
CPU, optimised bump	146.6	10.7×	—	—
CPU, adjoint (CoDiPack)	103.8	7.6×	26,900[a]	—
CPU, forward mode	78.0	5.7×	34,000[a]	—
CPU, analytic Greeks	20.3	1.5×	158,000[a]	**[HWiNFO]**[b]
FPGA, ZedBoard, 70 MHz (loop)	65.3	1.04×	15,300	44 μJ[c]
FPGA, ZCU102, 100 MHz (whole)	15.9	1.55×	62,800	40 μJ[c]

Cost: time relative to the price alone on the same platform. [a] 4 cores, 8 processes. [b] Measured package power under the 8-process load divided by throughput: **[P_{pkg} W / 158,000 s^{-1} = E μJ]**; single core **[P_1 W × 20.3 μs]**. [c] Engine power from simulated activity (A10): 0.676 W × 65.3 μs and 2.485 W × 15.9 μs.

Strike chains. All strikes of one expiry share the COS grid, so one characteristic function and one reverse sweep per term can serve every strike, with each extra strike costing about 7% of one option. This was scheduled and emulated (32 strikes in 4,892 cycles on the 64-bit design) but not built, and the same sharing helps a CPU (2.3 μs per strike at 32 strikes on one core), so it is not claimed as an advantage.

Limitations. (i) The reverse sweep is derived by hand (A13); generating it automatically from the forward graph is the natural next step. (ii) The bound is first order and covers rounding only (A6, Section VI-F). (iii) The FPGA energy is from simulated activity of the engine alone (A10), and the bus transfers are not timed (A9); a processor-based ZCU102 design that times every stage and reads the board's power monitors has been built but not yet run successfully. (iv) The Zynq-7020 design closes timing at 79 MHz, not 100 MHz. (v) The input domain is bounded (A1).

VIII. Conclusion

For the COS method, whose operations do not depend on the input, adjoint differentiation needs no tape: the reverse sweep can be compiled into the forward pass's datapath and fill

the multiplier slots the forward pass leaves idle. The resulting FPGA engine returns a Heston price and all nine Greeks for 1.04 times the cost of the price, 18.4× fewer cycles and 32–560× more accurate than bump-and-reprice on the same hardware, with a first-order rounding-error bound on every Greek that was never exceeded, and it ran bit-exact on two FPGA families, including on real market calibrations. Generating the reverse sweep automatically and placing several engines per device are the next steps.

REFERENCES

[1] S. L. Heston, "A closed-form solution for options with stochastic volatility with applications to bond and currency options," *Rev. Financial Stud.*, vol. 6, no. 2, pp. 327–343, 1993.

[2] F. Fang and C. W. Oosterlee, "A novel pricing method for European options based on Fourier-cosine series expansions," *SIAM J. Sci. Comput.*, vol. 31, no. 2, pp. 826–848, 2008.

[3] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert, "The little Heston trap," *Wilmott Mag.*, pp. 83–92, Jan. 2007.

[4] M. Giles and P. Glasserman, "Smoking adjoints: Fast Monte Carlo Greeks," *Risk*, vol. 19, pp. 88–92, Jan. 2006.

[5] L. Capriotti, "Fast Greeks by algorithmic differentiation," *J. Comput. Finance*, vol. 14, no. 3, pp. 3–35, 2011.

[6] L. Capriotti and M. Giles, "Fifteen years of adjoint algorithmic differentiation (AAD) in finance," *Quant. Finance*, vol. 24, no. 9, 2024.

[7] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA, USA: SIAM, 2008.

[8] S. Linnainmaa, "Taylor expansion of the accumulated rounding error," *BIT Numer. Math.*, vol. 16, no. 2, pp. 146–160, 1976.

[9] Y. Cui, S. del Baño Rollin, and G. Germano, "Full and fast calibration of the Heston stochastic volatility model," *Eur. J. Oper. Res.*, vol. 263, no. 2, pp. 625–638, 2017.

[10] M. Sagebaum, T. Albring, and N. R. Gauger, "High-performance derivative computations using CoDiPack," *ACM Trans. Math. Softw.*, vol. 45, no. 4, 2019.

[11] M. Klaisoongnoen, N. Brown, and O. Thomson Brown, "Low-power option Greeks: Efficiency-driven market risk analysis using FPGAs," in *Proc. Int. Symp. Highly-Efficient Accelerators and Reconfigurable Technologies (HEART)*, 2022.

[12] C. de Schryver *et al.*, "An energy efficient FPGA accelerator for Monte Carlo option pricing with the Heston model," in *Proc. Int. Conf. Reconfigurable Computing and FPGAs (ReConFig)*, 2011.

[13] AMD, "Vitis Libraries: Quantitative Finance Library," [Online]. Available: https://github.com/Xilinx/Vitis_Libraries

[14] F. Gremse, A. Höfter, L. Razik, F. Kiessling, and U. Naumann, "GPU-accelerated adjoint algorithmic differentiation," *Comput. Phys. Commun.*, vol. 200, pp. 300–311, 2016.

[15] P. T. P. Tang, "Table-driven implementation of the exponential function in IEEE floating-point arithmetic," *ACM Trans. Math. Softw.*, vol. 15, no. 2, pp. 144–157, 1989.

[16] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.

[17] B. R. Rau, "Iterative modulo scheduling: An algorithm for software pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO)*, 1994, pp. 63–74.